POS V3 EXE Build Manual

Mauritius POS V3 - Josea Outsourcing Ltd

Document version 1.1 - June 2026

Purpose

This manual explains how to package POS V3 Mauritius as a standalone Windows executable, prepare the distribution folder, and verify that the packaged app still supports the current production features: sales, VAT, stock, barcode tools, float, takings, backups, licensing, and optional billing.

Build Output

The build script creates a one-folder PyInstaller bundle.

```
dist\pos_v3\pos_v3.exe
```

Ship the full dist\pos_v3 folder, not only the EXE. The folder contains the executable plus bundled templates, static files, database assets, and runtime files copied after the build.

Prerequisites

- Windows 10 or Windows 11 build machine.
- Python 3.11 or 3.12.
- Project dependencies installed from the packaged requirements.txt.
- PyInstaller installed.
- Optional: PyArmor if using the -Obfuscate build option.
- Vendor private key kept on the vendor machine only.

```
pip install -r pos_v3_mauritius_professional_dashboard\requirements.txt
pip install pyinstaller
```

Standard Build

1. Open PowerShell in the repository root.
2. Activate the virtual environment.
3. Run the build script.
4. Wait for PyInstaller to complete.
5. Review the final dist\pos_v3 folder.

```
.\.venv\Scripts\Activate.ps1
.\build.ps1
```

Clean Build

Use a clean build when old PyInstaller output may be stale.

```
.\build.ps1 -Clean
```

The clean option removes build artifacts before packaging. Re-check dist\pos_v3 after the build finishes.

Optional Obfuscated Build

If PyArmor is installed and source obfuscation is required, run:

```
.\build.ps1 -Obfuscate
```

The build script obfuscates serve.py, app.py, license_manager.py, and keygen.py, then packages the obfuscated serve.py entry point.

Bundled Data and Hidden Imports

The PyInstaller configuration bundles templates, static assets, and database files. Hidden imports include Flask, Waitress, schedule, cryptography modules, python-barcode, pyzbar, Pillow, Stripe, SQLAlchemy, Alembic, pyodbc, email/smtp modules, sqlite3, uuid, subprocess, and platform.
Excluded modules are tkinter, matplotlib, numpy, and pandas to reduce package size.

Files to Include in Customer Distribution

- The complete dist\pos_v3 folder.
- start.ps1 copied by the build script.
- .env.example copied by the build script, if present.
- vendor_public.pem.
- A customer-specific license.key after the license is issued.
- Updated README.md, INSTALL.md, and this build manual when shipping staff documentation.

Files Never to Include

- vendor_private.pem.
- Local virtual environments.
- Build-machine backup archives.
- Test client databases unless intentionally migrating a client.
- Unencrypted client data exports.

License Preparation

1. On the customer machine, obtain the hardware fingerprint using keygen.py fingerprint or the License page.
2. On the vendor laptop, issue the license with keygen.py issue.
3. Copy the generated license file into dist\pos_v3.
4. Rename it to license.key.
5. Set LICENSE_ENFORCE=true and LICENSE_FILE=license.key in the runtime environment or start script.

```
python keygen.py issue --company "Client Name" --company-id "client-001" --days 365 --hardware-fp <fingerprint>
```

Runtime Configuration Checklist

- FLASK_SECRET_KEY is set to a strong random value.
- HOST is 0.0.0.0 when LAN access is required.
- PORT is the agreed local port, normally 8080.
- LICENSE_ENFORCE is true for client deployments.
- LICENSE_FILE points to license.key or an HTTPS license URL.
- APP_ENFORCE_BILLING is false unless Stripe billing is active.
- Email settings are configured in the app if backup or operational emails are required.

Post-Build Smoke Test

1. Start dist\pos_v3\pos_v3.exe or run start.ps1 from dist\pos_v3.
2. Open http://localhost:8080, or the configured host and port.
3. Log in with the admin account and change the default password if this is a client environment.
4. Confirm License shows valid details.
5. Create a test product with a SKU/barcode.
6. Open Barcodes and confirm a label can be generated.
7. Open Sales, scan or type the SKU, finalize a receipt, and print preview.
8. Record a purchase and confirm Stock updates.
9. Open Physical Stock and Stock Variance.
10. Record Float and Takings values.
11. Open Daily Sales, Daily Purchases, Daily Takings, Reports, and VAT.
12. Download an encrypted backup from Database Maintenance.
13. Confirm Audit Logs record the expected actions.

Troubleshooting

- If PyInstaller misses a module, add it to the HiddenImports list in build.ps1 and rebuild.
- If templates or styles are missing, confirm the --add-data entries include templates and static.
- If barcode decoding fails in the EXE, confirm pyzbar and its native dependencies are available in the packaged environment.
- If Stripe or SQL modules fail on startup, confirm the matching hidden imports remain in the build script.
- If the app opens but license fails, verify the fingerprint, license.key filename, and LICENSE_FILE setting.